CENTRO UNIVERSITÁRIO MAX PLANCK (UNIMAX)
CIÊNCIA DA COMPUTAÇÃO

GUILHERME DE FARIA GIMENES

A MESMA IDEIA, DOIS CONSTRUTORES: O QUE MUDA EM UM PRODUTO DE SOFTWARE QUANDO QUEM CONDUZ A IA ENTENDE, OU NÃO, DE ENGENHARIA E ARQUITETURA DE SOFTWARE

INDAIATUBA
2026

GUILHERME DE FARIA GIMENES

A MESMA IDEIA, DOIS CONSTRUTORES: O QUE MUDA EM UM PRODUTO DE SOFTWARE QUANDO QUEM CONDUZ A IA ENTENDE, OU NÃO, DE ENGENHARIA E ARQUITETURA DE SOFTWARE

Trabalho de Conclusão de Curso apresentado ao curso de Ciência da Computação do Centro Universitário Max Planck (UniMAX), como requisito parcial para a obtenção do título de Bacharel em Ciência da Computação.

Orientador: Prof. Luiz Claudio Chiavini Oliveira Junior.

INDAIATUBA

2026

A MESMA IDEIA, DOIS CONSTRUTORES: O QUE MUDA EM UM PRODUTO DE SOFTWARE QUANDO QUEM CONDUZ A IA ENTENDE, OU NÃO, DE ENGENHARIA E ARQUITETURA DE SOFTWARE

Resumo: Agentes de codificação baseados em modelos de linguagem permitem que uma pessoa sem formação técnica publique um aplicativo com login, fotos e dados pessoais em poucas horas, e a mesma ferramenta está na mão de engenheiros de software que passaram a trabalhar prompts, documentos de contexto e critérios de verificação em vez do código. Este trabalho mede o que muda no produto quando a mesma ideia é construída pelo mesmo agente, conduzido por um usuário leigo e por um engenheiro e arquiteto de software. Um experimento controlado construiu o Maxx, rede social gamificada para a UniMax, em duas trilhas independentes com o mesmo agente, o mesmo modelo, a mesma versão e a mesma validação, variando apenas quem pede e como pede. A persona leiga foi construída a partir de uma pesquisa anônima com 28 respondentes; a do engenheiro é o próprio autor. Os produtos foram avaliados em seis eixos por agentes revisores independentes e pelo autor, às cegas, com testes de referência e inventário de cobertura. Na trilha do leigo, o agente não fez nenhuma pergunta de produto em oito rodadas, decidiu sozinho a regra central do aplicativo no primeiro prompt e entregou, em três horas, um produto publicado com oito falhas de segurança reproduzíveis e cinco regras da ideia ausentes; na trilha do engenheiro, um protocolo que proíbe o agente de assumir decisões fora dos documentos produziu, em cerca de cinquenta horas, um produto com testes, integração contínua e registro de cada decisão. A diferença não está na capacidade do modelo, que escreveu código bom nos dois casos, mas na informação que chegou a ele no início e na verificação que o acompanhou depois.

Palavras-chave: Engenharia de Software; Agentes de codificação; Vibe coding; Inteligência artificial generativa; Qualidade de software.

The Same Idea, Two Builders: what changes in a software product when the person driving the AI does, or does not, understand software engineering and architecture

Abstract: Coding agents built on large language models allow a person with no technical background to publish an application with login, photos and personal data in a few hours, and the same tool is in the hands of software engineers who now work on prompts, context documents and verification criteria instead of code. This work measures what changes in the product when the same idea is built by the same agent, driven by a layperson and by a software engineer and architect. A controlled experiment built Maxx, a gamified social network for UniMax, in two independent tracks with the same agent, model, version and validation, varying only who asks and how. The lay persona was built from an anonymous survey with 28 respondents; the engineer persona is the author. The products were evaluated on six axes by independent reviewing agents and by the author, blind, with reference tests and a coverage inventory. In the lay track, the agent asked no product question in eight rounds, decided the application's central rule on its own in the first prompt, and delivered, in three hours, a published product with eight reproducible security flaws and five missing rules from the idea; in the engineer track, a protocol that forbids the agent from assuming decisions outside the documents produced, in about fifty hours, a product with tests, continuous integration and a record of every decision. The difference lies not in the model's capability, which wrote good code in both cases, but in the information that reached it at the start and the verification that followed.

Keywords: Software Engineering; Coding agents; Vibe coding; Generative artificial intelligence; Software quality.

SUMÁRIO

1 INTRODUÇÃO.....5
1.1 Contextualização.....5
1.2 Problema, pergunta e objetivos.....6
1.3 Justificativa e delimitação.....7
2 FUNDAMENTAÇÃO TEÓRICA.....8
2.1 Modelos de linguagem e vibe coding.....8
2.2 Agentes de codificação.....9
2.3 O trabalho de definição e a engenharia de prompt.....9
2.4 Qualidade de software: os seis eixos.....10
2.5 Trabalhos relacionados.....10
3 O EXPERIMENTO.....12
3.1 Desenho, ideia e personas.....12
3.2 Trilhas, protocolo e ambiente.....13
3.3 Validação e rubrica.....14
4 RESULTADOS.....16
4.1 O que cada trilha custou e produziu.....16
4.2 O pedido de abertura e a primeira decisão.....16
4.3 Decisões sem perguntar.....17
4.4 Verificação e segurança.....18
4.5 Avaliação por eixos.....19
5 DISCUSSÃO.....21
5.1 Pouca informação no início, produto errado no fim.....21
5.2 Autonomia sem verificação e riscos reais.....21
5.3 O que muda quando o operador sabe o que pedir.....22
5.4 Limites.....23
6 CONCLUSÃO.....24
REFERÊNCIAS.....25
APÊNDICE A: RUBRICA DE AVALIAÇÃO.....29
A.1 Experiência do usuário.....29
A.2 Segurança.....29
A.3 Infraestrutura.....30
A.4 Documentação.....30
A.5 Código.....31
A.6 Projeto e arquitetura.....31

1 INTRODUÇÃO

1.1 Contextualização

A área de tecnologia nunca exigiu, como condição de entrada, um diploma de curso superior. Autodidatas, técnicos e pessoas vindas de outras formações sempre entraram no mercado e nele permaneceram: na pesquisa anual do Stack Overflow, menos da metade dos desenvolvedores profissionais tem bacharelado e mais de 80% dizem ter aprendido por recursos online (STACK OVERFLOW, 2024); no Brasil, o gargalo apontado pela Brasscom não é a oferta de formados, mas a passagem para o emprego formal (BRASSCOM, 2026). Quem sabe fazer entra; o que mudou nos últimos anos é o que significa "saber fazer".

Essa mudança vem da capacidade dos modelos de linguagem de gerar código. Em outubro de 2024, o Google informou que mais de um quarto do código novo da empresa era gerado por IA e revisado por engenheiros, fração que passava de 30% em abril de 2025 (ENGADGET, 2024; OFFICECHAI, 2025); no mesmo mês, a Microsoft declarou que entre 20% e 30% do código dos seus repositórios era escrito por software (TECHCRUNCH, 2025a). Em janeiro de 2026, Linus Torvalds, criador do Linux, publicou um projeto pessoal cuja parte em Python foi, nas palavras dele, feita por vibe coding com uma ferramenta de agente, e ao longo do ano defendeu que o kernel aceitasse contribuições assistidas por IA, desde que declaradas e com um humano respondendo pelo código (IT'S FOSS, 2026; TOM'S HARDWARE, 2026). Cada vez menos quem coloca a mão no código é a pessoa; o profissional passa a precisar entender o produto que quer, descrever as regras, pensar a arquitetura e a infraestrutura e saber repassar isso a quem constrói, seja um time, seja um agente.

A consequência é que a barreira do código foi quebrada. Até pouco tempo, publicar um produto de software exigia alguém com experiência, e mesmo essa pessoa levava semanas ou meses. Hoje qualquer pessoa com acesso a um agente de codificação coloca um aplicativo no ar numa tarde, como este trabalho demonstra em uma de suas trilhas; a plataforma Lovable informou mais de 60 milhões de projetos criados em menos de dois anos, quase 80% deles como negócio ou projeto paralelo (SILICONANGLE, 2026). O termo que nomeia a prática, vibe coding, cunhado por Andrej Karpathy em fevereiro de 2025 para um jeito de programar em que a pessoa se entrega à conversa com a IA e esquece que o código existe, foi eleito palavra do ano de 2025 pelo dicionário Collins (ARS TECHNICA, 2025; CNN, 2025). Os assistentes de IA se tornaram cotidianos: o ChatGPT passou de 700 milhões de usuários semanais, com cerca de 70% do uso fora do trabalho (OPENAI, 2025); no Brasil, cerca de 50 milhões de pessoas usaram IA generativa em 2025 (CETIC.BR, 2025); entre desenvolvedores, 84% usavam ou pretendiam usar IA em 2025 e 73% dos que usam o

fazem diariamente em 2026, com o Claude Code e o GitHub Copilot como os mais citados (SD TIMES, 2025; STACK OVERFLOW, 2026). As empresas respondem com gasto, como os US$ 34,9 bilhões de infraestrutura da Microsoft num único trimestre de 2025 (SHERWOOD NEWS, 2025), e 88% das organizações já usam IA em ao menos uma função (MCKINSEY, 2025). Os desenvolvedores colocam cada vez menos a mão no código e trabalham cada vez mais o que dizem à IA.

1.2 Problema, pergunta e objetivos

Ao lado da adoção há um fenômeno menos visível: as pessoas usam pouco do que essas ferramentas oferecem e, com frequência, usam de forma errada, e isso é um problema sobretudo quando o que está sendo construído é um produto de software que outras pessoas vão usar com seus dados. Para verificar essa premissa, este trabalho começou com uma pesquisa anônima com 28 pessoas, em que cada uma escreveu, com as próprias palavras, como pediria a uma IA um aplicativo descrito num cenário. As mensagens de quem não desenvolve software têm em média 165 caracteres, nenhuma menciona segurança, teste, hospedagem ou publicação, e nenhuma descreve como a regra central do cenário, um sistema de pontos, deveria funcionar. Do outro lado estão os profissionais que passaram a trabalhar prompts, documentos de contexto e critérios de verificação em vez do código. A mesma ferramenta está na mão de quem tem formação em engenharia de software e de quem não tem, e não se sabe, com evidência empírica, o que muda no produto entre um caso e outro.

A pergunta de pesquisa é: o que muda no produto, em arquitetura, segurança, código, infraestrutura, documentação e experiência do usuário, quando a mesma ideia é construída com IA por um usuário leigo e por um engenheiro de software? A premissa posta à prova é que o trabalho de definição (requisitos, design, arquitetura, segurança, testes, operação) não desaparece quando a IA escreve o código: ou a pessoa faz antes, ou o agente faz por ela, com os padrões dele.

O objetivo geral é comparar o produto construído por um mesmo agente de IA, a partir da mesma ideia, quando conduzido por um usuário leigo e por um engenheiro e arquiteto de software, nos seis eixos de uma rubrica de qualidade. Os objetivos específicos são: caracterizar como pessoas sem formação técnica pedem software a uma IA; construir as duas trilhas sob protocolo controlado; avaliar cada produto com a rubrica, com testes de referência e com a avaliação cega do autor ao lado da de agentes revisores; medir quem fez o trabalho de definição, pelas decisões tomadas pelo agente sem pedido e pelas regras ausentes; e medir o custo de cada trilha.

1.3 Justificativa e delimitação

A pergunta importa porque os agentes de codificação tornam possível, a quem não programa, publicar um aplicativo com login, fotos e dados pessoais de terceiros. Se o que separa um produto seguro de um frágil é a disciplina de quem conduz, isso tem consequência para quem usa esses produtos, para quem ensina engenharia de software e para quem avalia código gerado por IA. Os relatórios do setor já mostram a escala do risco: 45% das amostras de código gerado por mais de cem modelos continham vulnerabilidades em 2025, e a taxa de aprovação em segurança não melhorou em 2026 (VERACODE, 2025; 2026); milhões de segredos vazam por ano em repositórios públicos (TECHRADAR, 2026); e incidentes recentes com aplicativos construídos por quem não programa expuseram dados pessoais em escala (ALBERGOTTI, 2025; WIZ, 2026; TECHCRUNCH, 2026). O que falta é evidência que isole o fator humano: mesma ideia, mesmo agente, operadores diferentes.

O recorte, um autor, uma ideia e um agente, é a força do desenho: a mesma pessoa conduz as duas trilhas, e só o conhecimento aplicado muda. A tese não afirma que o engenheiro tem ideias melhores; afirma que a disciplina de engenharia de software muda o que é construído a partir da mesma ideia, e mede onde e quanto. O experimento usa um agente (o Claude Code, da Anthropic), um modelo e uma versão fixados, um produto (o Maxx, rede social gamificada para a UniMax), um autor nas duas personas e uma execução por trilha; não compara modelos nem ferramentas, e a comparação de notas é descritiva, sem teste estatístico.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Modelos de linguagem e vibe coding

Modelos de linguagem de grande escala são sistemas treinados em grandes corpora de texto, capazes de gerar, completar e transformar texto, incluindo código. Zhao et al. (2023) destacam limitações que importam diretamente aqui: a geração é não determinística, o modelo não guarda memória entre sessões a não ser pelo que lhe é fornecido de novo, e há a alucinação, a produção de conteúdo plausível e incorreto. No código, Chen et al. (2021) mostraram que mesmo modelos de alto desempenho falham em cadeias longas de operações, o que exige validação sistemática do que é produzido. Essas propriedades aparecem no desenho do experimento: uma execução por condição limita a inferência, cada rodada é uma sessão nova com contexto controlado, e a alucinação é critério do eixo Código e objeto das "alegações sem lastro".

O termo vibe coding foi cunhado por Andrej Karpathy em fevereiro de 2025 para um jeito de programar em que a pessoa descreve o que quer em linguagem natural, aceita o que o modelo sugere sem ler o código e, quando algo quebra, cola a mensagem de erro de volta; ele mesmo o recomendava para projetos descartáveis (ARS TECHNICA, 2025). Do ponto de vista da engenharia de software, a prática desafia o que Pressman e Maxim (2016) e Sommerville (2018) colocam na base do processo, o levantamento de requisitos, o planejamento e a rastreabilidade das decisões. A questão deste trabalho não é se a prática funciona, e sim o que acontece com esses princípios quando quem conduz a conversa os conhece e quando não conhece.

2.2 Agentes de codificação

Um agente de codificação é um modelo de linguagem acoplado a ferramentas (ler e escrever arquivos, executar comandos, usar o git, buscar na web) e a um laço de controle que lhe permite planejar, agir, observar e agir de novo até dar a tarefa por terminada. O Claude Code, usado neste experimento, roda no terminal do usuário e expõe os controles que importam para o protocolo: modos de permissão (pedir aprovação antes de cada ação, executar sem pedir no modo auto, ou só planejar), arquivos de instrução persistentes chamados `CLAUDE.md`, lidos no início de cada sessão, memória automática entre sessões, que pode ser desligada, e transcrições completas (ANTHROPIC, 2026d; 2026e; 2026f). A documentação é explícita em dois pontos que o experimento usa: o `CLAUDE.md` é tratado pelo modelo como contexto, não como configuração imposta, e a revisão do código e dos comandos é responsabilidade de quem opera.

2.3 O trabalho de definição e a engenharia de prompt

Transformar uma ideia em algo construível é o que a engenharia de software chama de engenharia de requisitos: identificar atores, regras de negócio, requisitos não funcionais, casos de borda e critérios de aceite, e registrar as decisões (SOMMERVILLE, 2018). É esse trabalho que separa as personas deste experimento. A documentação do próprio fornecedor do agente descreve a consequência de pulá-lo: o diagnóstico mais comum de uma sessão ruim é o agente ter construído algo diferente do que o operador tinha em mente, por falta de informação que não está no repositório, e diante de um pedido vago o modelo tende a construir a menor coisa que o satisfaz (ANTHROPIC, 2026b). O guia de boas práticas pede que o modelo seja tratado como um funcionário brilhante e novo, sem contexto sobre as normas da equipe; recomenda explicar o porquê das instruções, dar exemplos, pedir um plano antes do código, e, quando não se sabe o que especificar, pedir que o agente entreviste o usuário até conseguir escrever uma especificação; e dar ao agente algo que ele possa rodar e que volte como passa ou falha, porque sem isso quem opera é a suíte de testes (ANTHROPIC, 2026a; 2026b). O trabalho da Anthropic sobre agentes confiáveis formula o mesmo em princípio: com menos supervisão há mais espaço para o agente ler mal a intenção, e ele deve pausar e perguntar quando incerto em vez de assumir (ANTHROPIC, 2026c). Este trabalho distingue técnica de prompt, aprendível em horas, de conhecimento do que perguntar, que é o que a formação dá; foi essa distinção que levou à decisão de os repositórios das duas trilhas nascerem vazios, sem especificação comum.

2.4 Qualidade de software: os seis eixos

A rubrica avalia seis eixos, ancorados no modelo de qualidade de produto da ISO/IEC 25010 e, no eixo de segurança, na lista OWASP Top 10 (ISO/IEC, 2011; OWASP, 2021), conforme o Quadro 1. A escala vai de 1 a 5 em cada eixo, com critérios escritos por nota. Quando o produto não implementa uma regra da ideia, o critério é marcado como ausente, não como errado: a nota é calculada sobre o que existe e a ausência vai para a análise como resultado, porque é exatamente o que se quer medir. Como a nota de um modelo usado como revisor tem vieses conhecidos (ZHENG et al., 2023), ela é tratada como rascunho: a nota do autor é a que vale, as duas são dadas às cegas e a divergência é registrada como dado.

Quadro 1 – Eixos da rubrica e características de qualidade correspondentes

Eixo	Característica (ISO/IEC 25010) e referência
1. Experiência do usuário	Usabilidade; referência funcional: especificação das seis telas
2. Segurança	Confidencialidade, integridade, responsabilização; OWASP Top 10
3. Infraestrutura	Confiabilidade e desempenho; deploy reproduzível e teste de carga
4. Documentação	Analisabilidade; registro de regras, decisões e execução
5. Código	Correção das regras, testabilidade, modularidade; testes de referência
6. Projeto e arquitetura	Modularidade, modificabilidade; coerência com a stack escolhida

Fonte: elaborado pelo autor (2026).

2.5 Trabalhos relacionados

Três grupos de estudos cercam a pergunta sem respondê-la. Em laboratório, a Veracode submeteu mais de cem modelos a oitenta tarefas e encontrou vulnerabilidades em 45% das amostras em 2025 e uma taxa de aprovação estagnada em torno de 56% em 2026 (VERACODE, 2025; 2026). Com pessoas, Perry et al. (2023), em experimento controlado com 47 participantes, mostraram que quem programava com assistente de IA escreveu código menos seguro e acreditou mais que ele era seguro, e que quem desconfiava e refinava os prompts produziu menos falhas. Em produção, a GitGuardian contou cerca de 29 milhões de segredos novos expostos no GitHub em 2025 (TECHRADAR, 2026); a Escape varreu mais de 5.600 aplicativos de plataformas de vibe coding e achou mais de 2.000 vulnerabilidades (ESCAPE, 2025); e a imprensa documentou aplicativos da Lovable expondo dados por falta de regras de acesso, a rede Moltbook com milhões de registros abertos, milhares de bancos Supabase públicos e um agente da Replit apagando um banco de produção (ALBERGOTTI, 2025; WIZ, 2026; TECHCRUNCH, 2026; NOLAN, 2025). Nenhum desses estudos isola o operador: os de laboratório não têm pessoa, o de Perry et al.

compara pessoas com e sem assistente, e os de produção medem o agregado sem saber quem pediu o quê. A lacuna que este trabalho ocupa é a comparação, sob protocolo, de duas pessoas com conhecimentos diferentes conduzindo o mesmo agente a partir da mesma ideia.

3 O EXPERIMENTO

3.1 Desenho, ideia e personas

O experimento constrói o mesmo aplicativo, o Maxx, em duas trilhas independentes, a partir da mesma ideia, com o mesmo agente de codificação, o mesmo modelo, a mesma versão, os mesmos revisores e a mesma validação. O que varia é quem pede e como pede. Um único agente e um único conjunto de revisores porque a pergunta é sobre quem conduz, não sobre qual modelo.

A ideia é a mesma nas duas trilhas e cabe num parágrafo de corredor: uma rede social para a UniMax, parecida com o Instagram mas sem stories, vídeos e mensagens, em que quem usa ganha prêmios pelo que faz e os professores dão medalhas, tudo contando para um ranking geral e um por curso, só para alunos e professores, com um mural para os avisos da faculdade. Os repositórios das duas trilhas nascem vazios, de um mesmo commit semente; a decisão de não entregar às duas um arquivo comum de regras veio de perceber que isso daria ao leigo uma especificação que ele nunca escreveria.

A persona do engenheiro fez três definições antes do código. A primeira foi o design: entre agosto e setembro de 2026, rascunhos à mão, wireframes, duas telas de alta fidelidade e a prancha de identidade, depois transformados em seis telas finais em tema claro e escuro e um sistema de design numa ferramenta de design da Anthropic, em três sessões paralelas que exploraram direções diferentes e convergiram; a revisão do material exportado encontrou um defeito de navegação que passara despercebido. A segunda definição foi o produto, e a terceira a técnica, ambas na rodada de abertura da trilha. As três custaram entre nove e dez horas antes do primeiro prompt de código, contra zero da persona leiga. Também faz parte do método do engenheiro conversar antes de pedir: a ideia foi discutida com um modelo de IA antes de os prompts de abertura serem fechados, e os prompts de design e de abertura foram rascunhados com apoio de um assistente de IA e revisados contra a documentação oficial da ferramenta. A persona leiga usa um aplicativo existente como referência ("tipo o Instagram") e pede a tela direto.

A persona leiga foi construída a partir de dados: 28 respostas a um formulário anônimo em que cada pessoa escrevia a primeira mensagem que mandaria a uma IA para pedir o aplicativo de um cenário com login, mural de avisos e sistema de pontos. Dez respondentes nunca trabalharam com tecnologia, treze em função próxima e cinco com desenvolvimento. A primeira mensagem de quem não desenvolve tem em média 165 caracteres, contra 566 entre quem desenvolve; só 8 das 23 citam algum dos três elementos que o cenário pedia; nenhuma menciona segurança, teste, hospedagem ou publicação; e nenhuma descreve como os pontos funcionariam.

Perguntadas como saberiam que o aplicativo funciona, 78% pediriam a outras pessoas que testassem antes de lançar: a preocupação existe, mas mora na validação, não na primeira mensagem. A amostra é de conveniência, provavelmente mais jovem e mais familiarizada com IA do que o público leigo médio. A persona do engenheiro é o próprio autor, engenheiro e arquiteto de software com quase sete anos de carreira. O Quadro 2 compara as duas, e a política de reação do leigo, pré-registrada, é o controle para o fato de um autor único não conseguir não saber o que sabe: ele pede a tela pelo nome comum, aceita o que o agente propõe, reclama só do que vê quebrado ou feio ao usar e não introduz regra de produto nova.

Quadro 2 – As duas personas

Dimensão	Leigo	Engenheiro
Vocabulário	Nome comum da tela; referência a app existente	Atores, regras, requisitos não funcionais, critérios de aceite
Tamanho do pedido	Uma frase a um parágrafo (abertura com 481 caracteres)	Páginas de produto e técnica, mais um plano por passo
Ordem de trabalho	Pede a tela direto	Design, produto e técnica antes do código; plano de etapas por passo
Resposta ao agente	"Faz do jeito que ficar melhor"	Restrição técnica; nunca decide fora dos documentos
Validação	Usa o produto e reclama do que vê	Revisão por etapa, capturas, testes, CI, testes de referência
Regra de ouro	Nenhuma	O agente não assume: decisão que não está nos documentos não foi tomada

Fonte: elaborado pelo autor (2026).

3.2 Trilhas, protocolo e ambiente

Cada trilha é um repositório próprio, nascido da mesma semente vazia; dois repositórios, e não duas branches, porque o agente executa `git` e poderia ler a outra trilha. Cada rodada é uma sessão nova do agente, numa branch da rodada, sem orquestrador: o único texto que chega ao agente é o da persona. O prompt é salvo num arquivo antes do envio e vai ao agente idêntico; perguntas do agente são respondidas no estilo da persona e nunca com decisão de produto fora da abertura. Na trilha do leigo nada é corrigido; na do engenheiro, a correção faz parte da persona e tem o custo registrado. Cada sessão é exportada e a transcrição completa é copiada; dela saem mensagens, chamadas de ferramenta, tokens e tempos. Por rodada fica um registro com as decisões que o agente tomou sem o prompt pedir, as perguntas e respostas, os turnos humanos e o tempo. A pesquisa com os respondentes foi anônima, nenhum dado real de aluno entra nos produtos, as credenciais de

infraestrutura nunca passam pelo chat do agente, e tudo o que o experimento produziu está num repositório público.

Na trilha do leigo são oito rodadas: a abertura e sete pedidos (entrar e cadastrar com o acesso da banca, feed, perfil, mural, troféus, ranking e publicação). Na trilha do engenheiro, a rodada de abertura produziu, em cinco dias de discussão com o agente, o documento de produto, o registro de decisões técnicas, o plano mestre de treze passos e o `CLAUDE.md` do repositório; cada passo seguinte é uma rodada, aberta por um prompt que relembra o que fecha o passo e pede um plano de etapas com entrega e critério de aceite antes de qualquer código. O `CLAUDE.md` fixa o protocolo que o agente segue em toda sessão: ler os documentos antes de agir, nenhum código antes do plano aprovado, revisão humana de cada etapa antes do commit, um commit por etapa e um PR por passo, e a regra que a persona considera primordial, nas palavras do próprio arquivo: "Decisão que não está nesses arquivos não foi tomada. Pergunte, não assuma." A partir do passo 3 a persona acrescentou a exigência de integração contínua verde em cada etapa.

O ambiente é o mesmo nas duas trilhas e foi conferido no início de cada sessão: a mesma interface de linha de comando (Claude Code 2.1.308, depois 2.1.316), o mesmo modelo da família Opus da Anthropic escolhido por nome (`claude-opus-5-5`, esforço `medium`, confirmados na transcrição ao fim de cada sessão), modo de permissão auto, memória automática desligada, nenhum arquivo de instrução fora dos repositórios e as mesmas habilidades e conectores visíveis ao agente, nenhum relacionado ao projeto.

3.3 Validação e rubrica

Dois instrumentos independentes do agente validam cada produto: uma suíte de testes de referência de caixa preta, por HTTP, com um adaptador por produto, que cobre as fronteiras da regra de pontos que o produto tiver, entradas inválidas e a ordenação do ranking, marcando "ausente" o que o produto não tem; e um teste de carga com k6, que faz parte do último passo da trilha do engenheiro. [A PREENCHER: resultados no engenheiro.]

A rubrica, reproduzida na íntegra no Apêndice A, é aplicada uma vez por produto, sobre o estado final do repositório, em duas camadas independentes, e as duas pontuam o mesmo objeto: o que existe no commit avaliado, com cada observação apontando para um arquivo, e não o processo que o produziu, que entra na análise pelos registros de rodada e pelas contagens descritas adiante. A primeira são seis sessões não interativas do mesmo agente, uma por eixo, sobre um checkout limpo e só de leitura, sem acesso ao review dos outros eixos; um ensaio com um `CLAUDE.md` isca provou que o arquivo do produto é lido como artefato e não como

instrução. Cada review traz nota, justificativa, evidências com arquivo e linha, falhas reproduzíveis, regras ausentes e decisões do agente. A segunda camada é o autor, que preenche um formulário com as seis notas antes de abrir qualquer review, commita o formulário e só então lê os reviews; onde a divergência é de dois pontos ou mais, escreve uma linha explicando, sem alterar a nota cega. Completam a avaliação a contagem, pelo autor, das decisões que o agente tomou sem perguntar, por tipo e por forma (declarada ou silenciosa); um inventário de cobertura com quinze itens da ideia, igual para os dois produtos, em que cada item é marcado como funcionando, parcial, quebrado, simulado ou ausente; as alegações sem lastro, o que o agente diz que está pronto e o inventário mostra que não está; e as medidas de custo extraídas das transcrições. Uma regra da ideia que o produto não implementa não reduz a nota do eixo: o eixo é pontuado sobre o que existe, e a ausência vai para o inventário. Por isso a qualidade de um produto se lê em dois números juntos, a nota do que foi construído e a cobertura do que foi pedido, e um produto pode ter nota alta em experiência do usuário e cinco regras ausentes ao mesmo tempo, sem contradição.

4 RESULTADOS

Os dados vêm dos registros por rodada, das transcrições das sessões, dos prompts e respostas do autor, dos reviews dos agentes avaliadores, do formulário cego do autor e do inventário de cobertura, todos no repositório público do TCC, referenciado pelo hash do commit. Os pontos que dependem da avaliação do produto do engenheiro aparecem com a marca [A PREENCHER].

4.1 O que cada trilha custou e produziu

Tabela 1 – Execuções das duas trilhas iterativas (medidas das transcrições)

Medida	Leigo (8 rodadas)	Engenheiro (passos 0 a 11)
Mensagens de conteúdo do autor	31	144
Mensagens do agente	599	7.351
Tokens de saída do agente	678.689	11.870.796
Tempo	3 h 21 min em uma noite, 2 h 03 min delas na publicação	cerca de 50 h de atividade em 12 sessões, mais cinco dias de discussão de produto e técnica no passo 0
Commits do agente	9	113
Perguntas de produto antes de construir	0	8 bloqueadores, cerca de 40 lacunas e 4 contradições só na primeira resposta do passo 0; um plano com perguntas por passo nos seguintes
Testes automáticos no fim	3 casos em um arquivo, sem CI	581 de domínio, 598 de integração, 489 Jest, 77 e2e, CI com seis jobs por push
Produto no ar	Sim, Azure App Service gratuito, sem verificação de e-mail	[A PREENCHER: deploy do passo 13]

Fonte: elaborado pelo autor (2026).

A diferença de escala é a primeira constatação: o engenheiro escreveu quase cinco vezes mais mensagens e o agente gastou dezessete vezes mais tokens de saída. É o que custa discutir regras, aprovar planos, revisar cada etapa, exigir testes e CI e registrar decisões. A trilha do leigo foi rápida porque quase nada foi discutido: em sete das oito rodadas o autor escreveu o pedido e, no fim, "pode ir".

4.2 O pedido de abertura e a primeira decisão

A abertura do leigo tem 481 caracteres:

> "Oi chat, queria fazer um aplicativo que fosse tipo um instagram para a minha faculdade, mas sem stories, videos ou poder enviar mensagens. O nome

> seria Maxx. Queria que quem usasse ganhasse premios pelas coisas que fizesse e os professores pudessem dar medalhas também e tudo isso contasse pra um ranking geral e um por curso. Seria só pra alunos e professores da faculdade UniMax. Ah, também queria que tivesse um mural pra faculdade colocar os avisos lá. Consegue fazer pra mim ?"

A abertura do engenheiro veio depois do design e das conversas prévias, e é uma mensagem de produto de várias páginas, seguida de uma mensagem técnica, que define os três tipos de usuário, o login por RA ou e-mail institucional, os dois tipos de prêmio, os tiers com faixas de pontos, a comprovação por foto com validação, a contestação, o ex-aluno e o mural por escopo. O agente respondeu a ela com oito bloqueadores, cerca de quarenta lacunas e quatro contradições antes de qualquer código.

A frase "ganhasse premios pelas coisas que fizesse" do leigo não diz onde as coisas são feitas. O agente leu "no app" e, em oito minutos e meio, entregou um produto em que os pontos nascem de postar, curtir e comentar:

```
// maxx-leigo, src/regras.js (rodada leigo_00_abertura)
const PONTOS = { post: 10, comentario: 3, curtida: 2 };
```

Nenhuma pergunta foi feita antes dessa escolha; ela foi declarada no relatório final, junto com outras treze decisões da mesma rodada (catálogo fixo de medalhas, conquistas automáticas, só alunos no ranking, login por domínio de e-mail sem verificação, senha padrão de administrador, app web em vez de app de celular, SQLite em arquivo). A ideia original, prêmios por participação em eventos e méritos em aula, não existe no produto do leigo porque ninguém a contou ao agente, e o agente não perguntou. Essa decisão de minuto zero determinou tudo o que veio depois: as sete rodadas seguintes construíram feed, perfil, mural, troféus, ranking e publicação sobre essa regra de pontos, e o inventário de cobertura do produto final mostra que, dos quinze itens da ideia, cinco estão ausentes (tipos de prêmio cadastrados pela instituição, tiers, professor só premiando aluno com quem tem aula, progressão a partir de prêmios, contestação e revogação), quatro parciais (perfil, mural sem escopos, comprovação sem revisão, registro sem decisões), um quebrado (ex-alunos) e um implementado mas desligado no ar (entrada da banca). Quase todas as ausências derivam daquela leitura inicial: sem prêmio por evento não há comprovação, fila de validação, contestação nem tiers.

4.3 Decisões sem perguntar

O autor contou 58 decisões tomadas pelo agente sem perguntar na trilha do leigo (38 declaradas no relatório da rodada, 20 silenciosas, visíveis só no que foi feito), em regra de produto (13), stack e arquitetura (7), modelo de dados (3), segurança (7),

UX e visual (15), testes e processo (3) e hospedagem e deploy (10). O agente avaliador de decisões, com critério mais fino, contou 168. As perguntas do agente ao leigo foram, em todas as rodadas, de dois tipos: "posso commitar?" e ofertas de próximo passo. Nenhuma pergunta de produto antes de construir. Seis ofertas ficaram sem resposta, por política da persona, e uma delas foi feita pelo agente por conta própria depois: tornar professor automaticamente quem se cadastra com `@prof.unieduk.com.br`, que viria a ser a principal falha de segurança apontada na avaliação.

Na trilha do engenheiro o padrão é o oposto, e não por iniciativa do modelo: o `CLAUDE.md` diz "Pergunte, não assuma" e exige um plano aprovado antes de qualquer código. No passo 11, o agente abriu com seis perguntas de produto e sete propostas declaradas; no passo 12, nove perguntas e três achados. Em cada relatório de etapa, as decisões tomadas dentro do que os documentos permitem aparecem listadas e entram no registro da rodada. O mesmo modelo, com a mesma versão e o mesmo modo de permissão, pergunta ou assume conforme o que o operador escreveu.

4.4 Verificação e segurança

Na trilha do leigo, o agente declarou duas vezes não ter aberto o aplicativo no navegador, pedindo ao autor que avisasse se algo ficasse estranho. Na rodada de publicação, duas alegações estavam erradas, "apaguei o post de teste" e "dados preservados após reinício", e só foram corrigidas porque a persona abriu o produto no celular e encontrou um post de teste com foto quebrada. Os testes do produto são três casos em um arquivo, sem integração contínua; a suíte de referência do experimento executou 51 casos contra ele: 43 passaram, 1 falhou e 7 ficaram ausentes porque o produto não tem tiers nem ex-aluno. O que o agente construiu, construiu bem; o problema está no que não foi pedido e no que não foi verificado. Na trilha do engenheiro a verificação é regra de processo: etapa só é dada como pronta com a CI verde, falha de CI vira commit `fix` separado, e cada etapa de tela termina com capturas nos dois temas que o autor confere contra as pranchetas. Nenhuma alegação do agente ficou sem um artefato que a sustente.

O agente avaliador de segurança deu nota 2 de 5 ao produto do leigo e listou oito falhas reproduzíveis apenas lendo o código. A principal está em duas linhas:

```
// maxx-leigo, src/config.js e src/app.js (cadastro)
DOMINIOS_PROFESSORES: ['prof.unieduk.com.br'],
...
config.DOMINIOS_PROFESSORES.includes(dominio) ? 'professor' : 'aluno'
```

O cadastro confere o formato e o domínio do e-mail digitado e não envia confirmação. Qualquer pessoa, de dentro ou de fora da faculdade, cria

`x@prof.unieduk.com.br` e entra como professor, com poder de dar medalhas que valem pontos a qualquer aluno, sem limite. As outras falhas seguem: contas falsas para inflar pontos, ocupação do e-mail de um colega, senha padrão de administrador no código e no README, login sem limite de tentativas, enumeração de contas, upload sem conferência de conteúdo e promoções feitas pela conta da banca que sobrevivem ao desligamento da banca. A base técnica é boa, e o revisor diz que sozinha chegaria perto de 4: SQL parametrizado, escape automático de HTML, senhas com scrypt, sessão com token aleatório. O modelo sabe escrever código seguro; o que faltou foi o desenho: quem é professor, como se prova a posse de um e-mail, o que acontece quando o produto fica público. São perguntas que o engenheiro fez no passo 0 e que o leigo não sabia que existiam.

A rodada de publicação acrescentou uma segunda camada de exposição: a persona colou o próprio e-mail institucional no chat quando o agente só precisava do domínio; o agente leu e exibiu o IP da rede local do autor; orientou duas vezes um leigo a entregar cartão de crédito a provedores de nuvem e sugeriu dado cadastral inexato para contornar um bloqueio; criou chave SSH fora do repositório, instalou software e obteve acesso autenticado à conta de nuvem do autor, tudo sem confirmação; gravou a senha do administrador em texto puro no disco; e deixou o site público sem verificação de e-mail. O modo auto do CLI pediu aprovação uma única vez em toda a trilha. Na trilha do engenheiro, as mesmas questões foram decididas em documento antes do código: RA nunca aparece para outros alunos, o e-mail vem de uma base abastecida pela faculdade, segredo nenhum no repositório, varredura de segredos na CI, credenciais de produção só no servidor. [A PREENCHER: nota de segurança do engenheiro.]

4.5 Avaliação por eixos

Tabela 2 – Notas do produto do leigo, trilha iterativa (1 a 5)

Eixo	Revisor	Autor	Divergência
1. Experiência do usuário	4	3	-1
2. Segurança	2	2	0
3. Infraestrutura	3	1	-2
4. Documentação	3	2	-1
5. Código	4	4	0
6. Projeto e arquitetura	3	1	-2
Total	19	13	

Fonte: elaborado pelo autor (2026).

As duas divergências de dois pontos têm a mesma explicação, registrada pelo autor depois de abrir os reviews, sem alterar a nota cega: o revisor pontuou o artefato (deploy público, configuração por variáveis, monólito coerente com a stack), sem acesso aos prompts e às sessões; o autor pontuou o processo que levou ao artefato, em que nenhuma discussão de arquitetura, regras, cenários, público ou volume de uso aconteceu, e em que a regra central do produto foi inventada pelo agente. Em UX, o revisor deu 4 porque as seis telas existem e funcionam de ponta a ponta; o autor deu 3 porque o produto "tem a cara de algo gerado rapidamente por uma IA" e porque os prêmios estão fora de contexto (Figura 1). Em Código, os dois deram 4.

Figura 1 – Produto do leigo aberto no celular da persona, logo depois da publicação



Fonte: registros do experimento, repositório público do TCC (2026).

[A PREENCHER: Tabela 3 com as notas do produto do engenheiro, testes de referência e inventário.]

5 DISCUSSÃO

5.1 Pouca informação no início, produto errado no fim

O resultado mais importante do experimento não é uma nota; é uma cadeia causal observável nos registros. O prompt de abertura do leigo tinha pouca informação. O agente, em vez de perguntar, preencheu as lacunas com a interpretação mais simples que satisfazia o texto. Essa decisão, tomada em minutos e declarada só no relatório final, se tornou a fundação do produto, e cada rodada seguinte foi construída sobre ela, com competência: o código é bom, as telas funcionam, o produto está no ar. O resultado é um produto estruturado, construído e entregue em torno de uma regra que a ideia original não tem.

A documentação da própria Anthropic descreve o mecanismo: o modelo como funcionário brilhante e novo, sem contexto; o agente que constrói algo diferente do que o operador tinha em mente por falta de informação que não está no repositório; e, diante de um pedido vago, a menor coisa que o satisfaz (ANTHROPIC, 2026a; 2026b). A abertura do leigo é o caso vago; a do engenheiro é o caso detalhado. A persona leiga não errou por descuido: ela não tinha como saber que "prêmios pelas coisas que fizesse" era ambíguo. Por isso a responsabilidade não pode ficar só com o operador. O mesmo guia recomenda, para quem não sabe o que especificar, pedir que o agente entreviste o usuário até conseguir escrever uma especificação, e o trabalho sobre agentes confiáveis afirma que o agente deve pausar e perguntar quando incerto (ANTHROPIC, 2026b; 2026c). Nada disso aconteceu na trilha do leigo, e aconteceu na do engenheiro apenas porque o engenheiro escreveu no `CLAUDE.md` que deveria acontecer.

5.2 Autonomia sem verificação e riscos reais

O agente do leigo disse duas vezes que não tinha olhado as telas; na publicação, afirmou duas coisas que não eram verdade e só se corrigiu quando a persona abriu o produto. O guia do Claude Code é direto: sem um teste que devolva passa ou falha, o operador é a suíte de testes, e a revisão do código e dos comandos é responsabilidade de quem opera (ANTHROPIC, 2026b; 2026d). O leigo, por definição, não consegue cumprir esse papel; o que ele consegue é usar o produto, e foi isso que pegou as duas alegações falsas. Tudo o que o uso não alcança ficou invisível até a avaliação. Perry et al. (2023) mostraram que quem programava com assistente de IA escreveu código menos seguro e acreditou mais que ele era seguro; a trilha do leigo reproduz isso em escala de produto, com relatório final confiante e nota 2 em segurança, e a do engenheiro reproduz o contrário, a desconfiança institucionalizada

em forma de CI, capturas, testes de referência e revisão por etapa. A Anthropic já alertava que agentes autônomos acumulam erros entre passos e recomendava pontos de parada humanos (SCHLUNTZ; ZHANG, 2024); na rodada de publicação, os únicos pontos de parada foram os que a persona impôs ao recusar pagar e ao recusar dar informação falsa.

As falhas do produto do leigo são as mesmas dos relatórios do setor e dos incidentes públicos: um backend sem controle de acesso adequado, exposto porque quem o pediu não sabia que havia algo a configurar. Em 2025, 170 de 1.645 aplicativos da vitrine da Lovable expunham dados por falta de regras de acesso (ALBERGOTTI, 2025); em 2026, a Moltbook deixou cerca de 4,75 milhões de registros abertos (WIZ, 2026) e cerca de 16 mil bancos Supabase com dados pessoais estavam públicos (TECHCRUNCH, 2026); um agente da Replit apagou um banco de produção e informou errado que o retorno não funcionaria (NOLAN, 2025), o mesmo gênero de alegação sem lastro que a persona pegou no post de teste. No Brasil, um artigo jurídico recente relaciona o software sem revisão às obrigações da LGPD (BARKOSKI, 2026), e o país registrou cerca de 7 milhões de contas vazadas no terceiro trimestre de 2026 (TI INSIDE, 2026). O produto do leigo, usado por uma faculdade de verdade, entraria nessa lista, e nenhuma dessas consequências é visível no momento em que a persona escreve "pode ir".

5.3 O que muda quando o operador sabe o que pedir

A comparação isola o operador: mesmo modelo, mesma versão, mesmo esforço, mesmo modo de permissão, mesma ideia, mesma máquina. O engenheiro não escreveu uma linha de código. Ele trouxe três coisas que a documentação da Anthropic nomeia e que o leigo não tinha: contexto que o agente não consegue ver (as regras do produto, o que "pronto" significa), uma forma de verificar (testes, CI, capturas, revisão por etapa) e uma regra de comportamento persistente no `CLAUDE.md`, a de não assumir. Tudo o mais no roteiro do engenheiro (documento de produto e registro de decisões antes do código, plano de etapas por passo, um commit por etapa, CI verde por etapa, PR por passo) existe para sustentar essa regra, e o agente a seguiu em praticamente todos os turnos. Isso tem um custo: dezessete vezes mais tokens, cerca de 50 horas contra três, e um autor capaz de ler planos, responder perguntas de domínio e conferir capturas. A trilha do engenheiro não é um atalho; é um método. O preço de não pagar esse custo não aparece no momento da construção, quando o leigo recebe um produto funcionando em oito minutos, mas quando alguém olha o que foi decidido e o que foi deixado aberto.

5.4 Limites

As duas personas foram interpretadas pelo mesmo autor, com a política de reação como controle. O agente da trilha do leigo leu, no histórico de merges, a estrutura em rodadas do experimento, um vazamento de metadado sem efeito observado no produto. Os agentes avaliadores trabalharam só com leitura e sem acesso às sessões, o que explica parte das divergências com o autor; a suíte de referência compensa isso em Código. A exigência de CI verde por etapa foi acrescentada no passo 3, e uma queda do GitHub Actions ocorreu durante o passo 12, ambas registradas. Com uma execução por trilha e notas ordinais, nada aqui é inferência estatística.

6 CONCLUSÃO

Com a mesma ideia e o mesmo agente, a persona leiga obteve em três horas um produto publicado que faz coisas diferentes das que foram pedidas, com oito falhas de segurança reproduzíveis e uma dezena de exposições ao longo do caminho, e a persona engenheira obteve, em cerca de cinquenta horas, um produto que faz o que os documentos dizem, com testes, CI e registro de cada decisão. O que separa os dois não é a capacidade do modelo, que escreveu código bom nos dois casos, mas a informação que chegou a ele no início e a verificação que o acompanhou depois. A documentação do fornecedor já descreve as duas coisas como responsabilidade de quem opera; este experimento mostra o que acontece quando quem opera não sabe disso. [A PREENCHER: comparação final das notas dos dois produtos.]

REFERÊNCIAS

ALBERGOTTI, Reed. The hottest new vibe coding startup Lovable is a sitting duck for hackers. Semafor, 29 maio 2025. Disponível em: https://www.semafor.com/article/05/29/2025/the-hottest-new-vibe-coding-startup-lovable-is-a-sitting-duck-for-hackers. Acesso em: 9 nov. 2026. (SEMAFOR, 2025)

ANTHROPIC. Prompting best practices. Claude Platform Docs, 2026a. Disponível em: https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices. Acesso em: 9 nov. 2026.

ANTHROPIC. Best practices for Claude Code. Claude Code Docs, 2026b. Disponível em: https://code.claude.com/docs/en/best-practices. Acesso em: 9 nov. 2026.

ANTHROPIC. Trustworthy agents in practice. 9 abr. 2026c. Disponível em: https://www.anthropic.com/research/trustworthy-agents. Acesso em: 9 nov. 2026.

ANTHROPIC. Security. Claude Code Docs, 2026d. Disponível em: https://code.claude.com/docs/en/security. Acesso em: 9 nov. 2026.

ANTHROPIC. How Claude remembers your project. Claude Code Docs, 2026e. Disponível em: https://code.claude.com/docs/en/memory. Acesso em: 9 nov. 2026.

ANTHROPIC. Choose a permission mode. Claude Code Docs, 2026f. Disponível em: https://code.claude.com/docs/en/permission-modes. Acesso em: 9 nov. 2026.

ARS TECHNICA. Is "vibe coding" with AI gnarly or reckless? Maybe some of both. 5 mar. 2025. Disponível em: https://arstechnica.com/ai/2025/03/is-vibe-coding-with-ai-gnarly-or-reckless-maybe-some-of-both/. Acesso em: 9 nov. 2026.

BARKOSKI, Lucas. Vibe coding e o custo jurídico do software sem revisão. Consultor Jurídico, 6 nov. 2026. Disponível em: https://conjur.com.br/2026-nov-06/vibe-coding-e-o-custo-juridico-do-software-sem-revisao/. Acesso em: 9 nov. 2026. (CONJUR, 2026)

BRASSCOM. Relatório Perspectivas do Mercado de Trabalho do Macrossetor de TIC. maio 2026. Disponível em: https://brasscom.org.br/wp-content/uploads/2026/07/Relatorio-Pespectivas-do-Mercado-de-Trabalho-Apresentacao-v9.pdf. Acesso em: 9 nov. 2026.

CETIC.BR. 50 milhões de brasileiros já usam IA, mas potenciais benefícios continuam limitados às camadas de maior renda e escolaridade. TIC Domicílios 2025, 9 dez. 2025. Disponível em: https://cetic.br/pt/noticia/50-milhoes-de-brasileiros-ja-usam-ia-mas-potenciais-beneficios-continuam-limitados-as-camadas-de-maior-renda-e-escolaridade/. Acesso em: 9 nov. 2026.

CHEN, Mark et al. Evaluating large language models trained on code. arXiv, 2021. Disponível em: https://arxiv.org/abs/2107.03374. Acesso em: 9 nov. 2026.

CNN. "Vibe coding" named Collins Dictionary's word of the year. 6 nov. 2025. Disponível em: https://kvia.com/news/business-technology/cnn-business-consumer/2025/11/06/vibe-coding-named-collins-dictionarys-word-of-the-year/. Acesso em: 9 nov. 2026.

ENGADGET. Google CEO says a quarter of the company's new code is already AI-generated. 30 out. 2024. Disponível em: https://www.engadget.com/ai/google-ceo-says-a-quarter-of-the-companys-new-code-is-already-ai-generated-180038896.html. Acesso em: 9 nov. 2026.

ENGADGET. Mark Zuckerberg predicts AI will write most of Meta's code within 12 to 18 months. 29 abr. 2025. Disponível em: https://engadget.com/ai/mark-zuckerberg-predicts-ai-will-write-most-of-metas-code-within-12-to-18-months-213851646.html. Acesso em: 9 nov. 2026.

ESCAPE. How we discovered vulnerabilities in apps built with vibe coding. 29 out. 2025. Disponível em: https://escape.tech/blog/methodology-how-we-discovered-vulnerabilities-apps-built-with-vibe-coding/. Acesso em: 9 nov. 2026.

FORTUNE. Meta raises 2026 AI spending outlook to as much as $145 billion. 29 abr. 2026. Disponível em: https://fortune.com/2026/04/29/meta-zuckerberg-145-billion-ai-spending-roi/. Acesso em: 9 nov. 2026.

ISO/IEC. ISO/IEC 25010:2011. Systems and software engineering: Systems and software Quality Requirements and Evaluation (SQuaRE): System and software quality models. Genebra: ISO, 2011.

IT'S FOSS. Linus Torvalds vibe-coded a Python visualizer with Google Antigravity. 11 jan. 2026. Disponível em: https://itsfoss.com/news/linus-torvalds-vibe-coding/. Acesso em: 9 nov. 2026.

MCKINSEY. The state of AI in 2025. 5 nov. 2025. Disponível em: https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai-2025. Acesso em: 9 nov. 2026.

NOLAN, Beatrice. AI coding tool Replit wiped a database and called it a catastrophic failure. Fortune, 23 jul. 2025. Disponível em: https://fortune.com/2025/07/23/ai-coding-tool-replit-wiped-database-called-it-a-catastrophic-failure/. Acesso em: 9 nov. 2026. (FORTUNE, 2025)

OFFICECHAI. Well over 30% of code at Google is now written by AI, says CEO Sundar Pichai. abr. 2025. Disponível em: https://officechai.com/ai/well-over-30-of-code-at-google-is-now-written-by-ai-ceo-sundar-pichai/. Acesso em: 9 nov. 2026.

OPENAI. How people are using ChatGPT. 15 set. 2025. Disponível em: https://openai.com/index/how-people-are-using-chatgpt/. Acesso em: 9 nov. 2026.

OWASP. OWASP Top 10: 2021. Disponível em: https://owasp.org/Top10/. Acesso em: 9 nov. 2026.

PERRY, Neil; SRIVASTAVA, Megha; KUMAR, Deepak; BONEH, Dan. Do users write more insecure code with AI assistants? In: ACM SIGSAC CONFERENCE ON COMPUTER AND COMMUNICATIONS SECURITY (CCS), 2023. Proceedings. New York: ACM, 2023. Disponível em: https://arxiv.org/abs/2211.03622. Acesso em: 9 nov. 2026.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de software: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

SCHLUNTZ, Erik; ZHANG, Barry. Building effective agents. Anthropic, 19 dez. 2024. Disponível em: https://www.anthropic.com/engineering/building-effective-agents. Acesso em: 9 nov. 2026.

SD TIMES. Stack Overflow: developers' trust in AI outputs is worsening year over year. 29 jul. 2025. Disponível em: https://sdtimes.com/softwaredev/stack-overflow-developers-trust-in-ai-outputs-is-worsening-year-over-year/. Acesso em: 9 nov. 2026.

SHERWOOD NEWS. Microsoft capex spending hits a record. 30 out. 2025. Disponível em: https://www.sherwood.news/tech/microsoft-capex-spending-2026. Acesso em: 9 nov. 2026.

SILICONANGLE. Vibe coding startup Lovable doubles valuation to $13.3B with $400M raise. 12 ago. 2026. Disponível em: https://siliconangle.com/2026/08/12/vibe-coding-startup-lovable-doubles-valuation-13-3b-400m-raise/. Acesso em: 9 nov. 2026.

SOMMERVILLE, Ian. Engenharia de software. 10. ed. São Paulo: Pearson, 2018.

STACK OVERFLOW. 2024 Developer Survey: developer profile. 2024. Disponível em: https://survey.stackoverflow.co/2024/developer-profile/. Acesso em: 9 nov. 2026.

STACK OVERFLOW. The results of the 2026 Developer Survey are here. 6 out. 2026. Disponível em: https://stackoverflow.blog/2026/10/06/the-results-of-the-2026-developer-survey-are-here/. Acesso em: 9 nov. 2026.

TECHCRUNCH. Microsoft CEO says up to 30% of the company's code was written by AI. 29 abr. 2025a. Disponível em: https://techcrunch.com/2025/04/29/microsoft-ceo-says-up-to-30-of-the-companys-code-was-written-by-ai/. Acesso em: 9 nov. 2026.

TECHCRUNCH. GitHub Copilot crosses 20 million all-time users. 30 jul. 2025b. Disponível em: https://techcrunch.com/2025/07/30/github-copilot-crosses-20-million-all-time-users/. Acesso em: 9 nov. 2026.

TECHCRUNCH. Some Supabase customers are publicly exposing reams of people's data to the web. 25 set. 2026. Disponível em:

https://techcrunch.com/2026/09/25/some-supabase-customers-are-publicly-exposing-reams-of-peoples-data-to-the-web/. Acesso em: 9 nov. 2026.

TECHRADAR. Over 29 million secrets were leaked on GitHub in 2025, and AI really isn't helping. mar. 2026. Disponível em: https://www.techradar.com/pro/security/over-29-million-secrets-were-leaked-on-github-in-2025-and-ai-really-isnt-helping. Acesso em: 9 nov. 2026.

TI INSIDE. Vazamentos de dados no Brasil crescem 21,5% e chegam a 7 milhões de contas. 4 nov. 2026. Disponível em: https://tiinside.com.br/04/11/2026/vazamentos-de-dados-no-brasil-crescem-215-e-chegam-a-7-milhoes-de-contas/. Acesso em: 9 nov. 2026.

TOM'S HARDWARE. Linux lays down the law on AI-generated code. 12 abr. 2026. Disponível em: https://www.tomshardware.com/software/linux/linux-lays-down-the-law-on-ai-generated-code-yes-to-copilot-no-to-ai-slop-and-humans-take-the-fall-for-mistakes-after-months-of-fierce-debate-torvalds-and-maintainers-come-to-an-agreement. Acesso em: 9 nov. 2026.

VERACODE. 2025 GenAI Code Security Report. 30 jul. 2025. Disponível em: https://www.veracode.com/press-release/ai-generated-code-poses-major-security-risks-in-nearly-half-of-all-development-tasks-veracode-research-reveals/. Acesso em: 9 nov. 2026.

VERACODE. 2026 GenAI Code Security Report. 28 jul. 2026. Disponível em: https://www.veracode.com/blog/2026-genai-code-security-report-ai-risk. Acesso em: 9 nov. 2026.

WIZ. Exposed Moltbook database reveals millions of API keys. 2 fev. 2026. Disponível em: https://www.wiz.io/blog/exposed-moltbook-database-reveals-millions-of-api-keys. Acesso em: 9 nov. 2026.

ZHAO, Wayne Xin et al. A survey of large language models. arXiv, 2023. Disponível em: https://arxiv.org/abs/2303.18223. Acesso em: 9 nov. 2026.

ZHENG, Lianmin et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS, 36., 2023. Disponível em: https://arxiv.org/abs/2306.05685. Acesso em: 9 nov. 2026.

APÊNDICE A: RUBRICA DE AVALIAÇÃO

Rubrica única, aplicada aos dois produtos pelos mesmos revisores (mesmo modelo, mesma versão, mesmo prompt por eixo) e pelo autor. Escala de 1 (crítico) a 5 (excelente) por eixo, total possível 30. Uma regra da ideia que o produto não implementa é marcada como ausente, não como errada: a nota do eixo é calculada sobre o que existe, e a ausência vai para o inventário de cobertura. Em todos os eixos o revisor lista, além da nota, as decisões que o agente tomou sem o prompt pedir; elas não entram na nota.

A.1 Experiência do usuário

Usabilidade, navegação, clareza dos fluxos, estados de erro e de carregamento, coerência visual. Referência funcional para as duas trilhas: a especificação das seis telas; os mockups do engenheiro são referência adicional só na trilha dele.

Quadro A.1. Critérios do eixo Experiência do usuário

Nota	Critério
1	Fluxos principais não funcionam ou não correspondem às telas pedidas (login sem RA, feed sem curtir ou comentar, ranking inexistente).
2	Fluxos existem, mas com passos desnecessários, navegação confusa ou estados de erro e carregamento não tratados.
3	As seis telas são utilizáveis de ponta a ponta; divergências de detalhe e inconsistências visuais entre telas.
4	Navegação clara, estados tratados, consistência visual; fiel à especificação (e, no engenheiro, aos mockups).
5	Idem ao 4, e resolve o que a especificação não diz de forma sensata e consistente; um membro da banca usa sem explicação.

Fonte: elaborado pelo autor (2026).

Fonte: elaborado pelo autor (2026).

A.2 Segurança

Brechas, falhas, bugs exploráveis, exposição de dados sensíveis (RA, e-mail, fotos, notas e pontos), autenticação e autorização.

Quadro A.2. Critérios do eixo Segurança

Nota	Critério
1	Segredos no código; sem validação de entrada; vulnerabilidade óbvia e explorável (injeção, dado de outro aluno alterável via requisição).
2	Validação incompleta; controle de acesso frágil (aluno chama rota de professor; RA de terceiro visível).

Nota	Critério
3	Validações básicas e autenticação funcionais, mas faltam casos de borda (quem pode dar medalha a quem, upload sem limite, sessão sem expiração).
4	Validação de entrada e autorização por papel e escopo consistentes; dados sensíveis protegidos; poucos pontos a reforçar.
5	Idem ao 4, com menor privilégio aplicado, segredos fora do código, proteção de upload e de sessão, e nenhuma falha reproduzível encontrada pelo revisor.

Fonte: elaborado pelo autor (2026).

Fonte: elaborado pelo autor (2026).

A.3 Infraestrutura

Deploy, preparação da estrutura para rodar fora da máquina do autor, suporte a muitos usuários, redundância, observabilidade mínima.

Quadro A.3. Critérios do eixo Infraestrutura

Nota	Critério
1	Não sobe fora da máquina onde foi gerado; sem instrução de execução; dependências não declaradas.
2	Sobe localmente com instruções, mas sem deploy ou com deploy manual e frágil; configuração misturada com código.
3	Deploy funcionando numa URL, configuração por ambiente, mas sem pensar em carga, falha ou recuperação.
4	Deploy reproduzível (contêineres ou equivalente), configuração por variável de ambiente, banco separado da aplicação, backup previsto.
5	Idem ao 4, com estrutura que aguenta crescimento (API sem estado, fotos fora da API, fila para o que pode esperar), redundância onde importa e teste de carga executado.

Fonte: elaborado pelo autor (2026).

Fonte: elaborado pelo autor (2026).

A.4 Documentação

Planos, documentos de produto e de arquitetura, padrões declarados, registro de decisões; o que uma pessoa nova precisaria ler para continuar o projeto.

Quadro A.4. Critérios do eixo Documentação

Nota	Critério
1	Nenhum documento além do código; não há como saber o que o produto deveria fazer.
2	README genérico ou gerado sem revisão; sem registro de decisões nem de regras de produto.
3	Regras de produto e instruções de execução documentadas; arquitetura só implícita no código.

Nota	Critério
4	Produto, arquitetura, convenções e decisões documentados e coerentes com o código; uma pessoa nova consegue continuar.
5	Idem ao 4, com decisões registradas com alternativas e consequências (ADR ou equivalente), plano de evolução e documentação mantida em dia com o que foi construído.

Fonte: elaborado pelo autor (2026).

Fonte: elaborado pelo autor (2026).

A.5 Código

Código limpo, sem duplicação, testado, com padrões de projeto onde cabem, legível e fácil de manter. Critério objetivo: rodar a suíte de testes de referência (fronteiras 30/31, 70/71, 100/101 e 150/151 da pontuação por tier; entradas inválidas; ordenação do ranking) antes de pontuar.

Quadro A.5. Critérios do eixo Código

Nota	Critério
1	Não compila ou não roda; lógica claramente errada (faixas de tier erradas, ranking fora de ordem); sinais de alucinação (API ou import inexistente).
2	Roda, mas falha nos testes de referência de fronteira; duplicação extensa; sem testes próprios.
3	Passa nos testes de fronteira; não trata entradas inválidas; testes próprios escassos; nomes e organização aceitáveis.
4	Passa em fronteira e entradas inválidas; testes cobrindo a lógica de negócio; sem duplicação relevante; convenções consistentes.
5	Idem ao 4, com padrões de projeto usados onde ajudam (e não onde não ajudam), código que se lê sem comentário e suíte que roda na CI.

Fonte: elaborado pelo autor (2026).

Fonte: elaborado pelo autor (2026).

A.6 Projeto e arquitetura

Estrutura do projeto e das páginas, separação de responsabilidades, escalabilidade, manutenibilidade e fidelidade à ideia: o quanto do que foi construído corresponde ao pitch e às regras que a persona pediu. A stack é decisão da trilha e é registrada como resultado, não pontuada; o que se pontua é a coerência com ela.

Quadro A.6. Critérios do eixo Projeto e arquitetura

Nota	Critério
1	Sem separação de camadas; estrutura que não permite localizar onde uma regra vive; produto distante do pitch.

Nota	Critério
2	Estrutura confusa, responsabilidades misturadas (regra de negócio na tela), decisões incoerentes com a própria stack escolhida.
3	Separação básica (interface, API, dados); acoplamentos evitáveis; alterar uma regra exige mexer em vários lugares; produto cobre o pitch com lacunas.
4	Camadas bem definidas, regras de produto localizadas num lugar só, estrutura preparada para novas telas sem retrabalho; produto fiel ao pitch e às regras pedidas.
5	Idem ao 4, com decisões justificadas, escalabilidade pensada para o campus inteiro e modelo de dados que representa as regras do produto (tiers, períodos, papéis) sem gambiarra.

Fonte: elaborado pelo autor (2026).

Fonte: elaborado pelo autor (2026).